**MTL783: Theory of Computation** 1st Semester, 2025-2026

Lecture 14 — 18/09/2025

*Lecturer: Prof Minati De* *Scribe: Team 14*

## Scribed by:


1. Hitesh Yadav (2022MT11322)
2. Sanya Sachan (2022MT11286)
3. Aayushi Singh (2022MT11955)
4. Ayush Pal (2022MT11282)
5. Samay Jain (2022MT11256)


## 1 Overview

So far we have discussed the following :

- **Regular Languages**: Recognized by **Finite Automata** (FA), which include both **Deterministic Finite Automata (DFA)** and **Nondeterministic Finite Automata (NFA)**.
  - FAs have **no memory** beyond their current state. They cannot count or remember unbounded information.
  - Regular languages can be described using **Regular Expressions**.
- **Context-Free Languages (CFLs)**: A strict superset of regular languages. Recognized by **Pushdown Automata (PDA)**.
  - PDAs are more powerful than FAs because they include a **stack** (a type of memory that allows unbounded push and pop operations).
  - CFLs can be generated by a **Context-Free Grammar (CFG)**.
  - Every context-free language can be recognized by a **Nondeterministic PDA (NPDA)**.
  - Some CFLs cannot be recognized by any **Deterministic PDA (DPDA)**.
- To show that a language is:
  - **Regular**, one typically provides a **regular expression**, **DFA**, or **NFA**.
  - **Context-free**, one typically provides a **CFG** or defines a **PDA** that recognizes it.

The **Pumping Lemma for Regular Languages** is a fundamental tool used to prove that certain languages are *not* regular. It provides a necessary property that all regular languages must satisfy.

**Statement:** If a language $L$ is regular, then there exists an integer $p \geq 1$ (called the **pumping length**) such that for every string $s \in L$ with $|s| \geq p$, there exist strings $x$, $y$, and $z$ such that:

1. $s = xyz$
2. $|y| > 0$ (the "pumped" part is non-empty)
3. $|xy| \leq p$ (the repetition occurs within the first $p$ symbols)
4. For all $i \geq 0$, the string $xy^iz \in L$

The contrapositive of this lemma is often used to **prove that a language is not regular** by assuming it is regular and showing that no matter how the string is decomposed into $x$, $y$, and $z$, one of the conditions is violated.

# 2 Pumping Lemma for Context-Free Languages (CFL)

## 2.1 Purpose and Application

The **Pumping Lemma for Context-Free Languages** serves a similar purpose to its regular counterpart: to prove that a given language is *not* context-free. It describes a fundamental property that all CFLs must possess. If a language does not satisfy this property, it cannot be context-free.

## 2.2 Formal Statement

If $L$ is a **Context-Free Language**, then there exists some integer $\mathbf{p}$ (the **pumping length**) such that any string $S \in L$ with length $|S| \geq p$ can be divided into five parts $S = uvxyz$ satisfying the following conditions:

1. For all $i \geq 0$, the string $uv^ixy^iz$ is in $L$.
2. $|vy| > 0$ (at least one of $v$ or $y$ must be non-empty).
3. $|vxy| \leq p$ (the combined length of $v$, $x$, and $y$ must be less than or equal to the pumping length $p$).

## 2.3 Intuition and Proof Sketch (Parse Trees)

The intuition behind the CFL Pumping Lemma comes from the structure of **parse trees** in **Context-Free Grammars** (CFGs). A CFG consists of:

- Variables (non-terminals)
- Terminals (alphabet)
- A start variable
- Production rules

A string $S$ is generated by starting from the **start variable** and applying production rules until only terminals remain. This process can be visualized as a **parse tree**, where the root is the start variable, intermediate nodes are variables, and leaves are terminals (which form the generated string when read from left to right).

Consider a parse tree for a string $S$. If the string $S$ is sufficiently long (i.e., $|S| \geq p$), then by the **Pigeonhole Principle**, any path from the root to a leaf must contain a repeated variable.

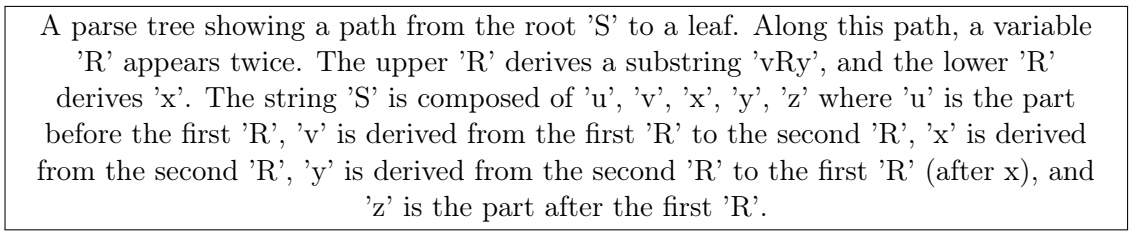


Figure 1: Visual representation of a parse tree for Pumping Lemma intuition.

Let's assume a grammar generates a string $S$. If there's a path in the parse tree where a variable, say $R$, repeats:

$$S \Rightarrow^* uRz$$
$$R \Rightarrow^* vRy$$
$$R \Rightarrow^* x$$

Then, we can substitute $R \Rightarrow^* vRy$ multiple times (pump up) or $R \Rightarrow^* x$ (pump down) to generate new strings:

- $S \Rightarrow^* uRz \Rightarrow^* uvRyz \Rightarrow^* uvvRyyz \Rightarrow^* uv^iRy^iz \Rightarrow^* uv^ixy^iz$ (pumping up)
- $S \Rightarrow^* uRz \Rightarrow^* uxz$ (pumping down, $i = 0$)

This allows us to "pump" the $v$ and $y$ parts of the string.

> **P.S.:** *The choice of $p$ is crucial. If a path from the root to a leaf has at least $|V| + 1$ intermediate nodes (variables), then a variable must repeat. This repetition is what enables the pumping action.*

### 2.3.1 Determining Pumping Length $p$

The **pumping length** $p$ for a CFL is related to the number of variables in the grammar and the maximum length of the right-hand side of any production rule.

Let $V$ be the set of variables in the CFG, and let $|V|$ be the number of variables. Let $B$ be the maximum length of the right-hand side of any production rule in the CFG. For example, if the grammar is in **Chomsky Normal Form** (CNF), $B$ would be 2.

Consider a parse tree. Each internal node corresponds to a variable. If a path in the parse tree has a length of at least $|V| + 1$ (number of variables + 1), then by the **Pigeonhole Principle**, at least one variable must repeat on that path.

The maximum number of children an internal node can have is $B$. The maximum number of leaves (terminals) that can be generated from a path of length $k$ is $B^k$.

The pumping length $p$ is chosen such that any string $S$ with $|S| \geq p$ guarantees that its parse tree (specifically, the one with the minimum number of internal nodes) will have a path with a repeated variable. A suitable choice for $p$ is $B^{|V|+2}$. This ensures that if a string's length is at least $p$, its parse tree must contain a path from the root to a leaf with at least $|V| + 2$ variable nodes. This guarantees a repeated variable.

**Conditions Explained:**

1. $|vy| > 0$: This condition ensures that we are actually pumping something. If $v$ and $y$ were both empty, pumping would have no effect. This is guaranteed by choosing the parse tree with the *minimum* number of internal nodes. If $v$ and $y$ were both empty, it would imply that the repeated variable $R$ could derive $x$ directly, leading to a smaller parse tree, contradicting the assumption of a minimum internal node count.

2. $|vxy| \leq p$: This condition is a bit trickier. It ensures that the pumped segment is "local" within the string. When walking up the longest path from a leaf towards the root, the first repetition of a variable (say, $R$) will define the $v, x, y$ segments. The sub-tree rooted at the higher $R$ (which generates $vxy$) will have a height bounded by $|V| + 1$. Since each node can have at most $B$ children, the length of the string derived from this sub-tree ($|vxy|$) will be at most $B^{|V|+1}$, which is less than or equal to $p$.

# 3 Applications of CFL Pumping Lemma (Proving Non-Context-Free Languages)

The general strategy to prove a language $L$ is *not* context-free using the Pumping Lemma is:

1. **Assume $L$ is context-free.**
2. By the Pumping Lemma, there exists a pumping length $p$.
3. **Choose a specific string** $S \in L$ such that $|S| \geq p$. This is the most critical step and often requires careful selection.
4. Show that no matter how $S$ is divided into $uvxyz$ satisfying the conditions ($|vy| > 0$ and $|vxy| \leq p$), pumping $uv^ixy^iz$ for some $i \neq 1$ results in a string that is *not* in $L$.
5. This leads to a contradiction, thus proving the initial assumption (that $L$ is context-free) is false.

## 3.1 Example: $L_2 = \{a^n b^n c^n \mid n \geq 0\}$

**Goal:** Prove $L_2$ is not context-free.

1. **Assume $L_2$ is context-free.**

2. Let $p$ be the pumping length.

3. **Choose string $S = a^p b^p c^p$.** Clearly, $S \in L_2$ and $|S| = 3p \geq p$.

4. According to the Pumping Lemma, $S$ can be divided into $uvxyz$ such that $|vy| > 0$ and $|vxy| \leq p$. Since $|vxy| \leq p$, the segment $vxy$ can only contain:

   - Only $a$'s (e.g., $a...a$)
   - Only $b$'s (e.g., $b...b$)
   - Only $c$'s (e.g., $c...c$)
   - A mix of $a$'s and $b$'s (e.g., $a...ab...b$)
   - A mix of $b$'s and $c$'s (e.g., $b...bc...c$)
   - It *cannot* contain $a$'s, $b$'s, *and* $c$'s because its length is at most $p$, and the string $a^p b^p c^p$ has $p$ $a$'s, $p$ $b$'s, and $p$ $c$'s. Any segment spanning all three would be longer than $p$.

   Now, consider pumping $uv^i xy^i z$:

   - **Case 1: $vxy$ contains only one type of character (e.g., only $a$'s).** If $vxy$ is within the $a$ block, then $v$ and $y$ consist only of $a$'s. Pumping up ($i = 2$) would increase the number of $a$'s, but $b$'s and $c$'s would remain $p$. The resulting string $a^{p+|v|+|y|} b^p c^p$ would have an unequal number of $a$'s, $b$'s, and $c$'s, so it's not in $L_2$. Pumping down ($i = 0$) would decrease the number of $a$'s, leading to a similar contradiction.
   - **Case 2: $vxy$ contains two types of characters (e.g., $a$'s and $b$'s).** If $vxy$ spans the $a$ and $b$ boundary (e.g., $a...ab...b$), then $v$ and $y$ together must contain $a$'s and/or $b$'s. Pumping up ($i = 2$) would increase the count of $a$'s and/or $b$'s, but the count of $c$'s would remain $p$. The resulting string would not have an equal number of $a$'s, $b$'s, and $c$'s, so it's not in $L_2$.

5. In all possible cases, pumping $S$ leads to a string not in $L_2$. This contradicts the Pumping Lemma. Therefore, $L_2$ is not context-free.

**P.S.:** *The Pumping Lemma is a one-way street. If a language satisfies the Pumping Lemma, it might be context-free, but it's not guaranteed. If it doesn't satisfy the Pumping Lemma, then it's definitely not context-free. You cannot use the Pumping Lemma to prove a language is context-free; for that, a CFG or PDA must be provided.*

### 3.2 Example: $L_3 = \{a^i b^j c^k \mid 0 \leq i \leq j \leq k\}$

**Goal:** Prove $L_3$ is not context-free.

1. **Assume $L_3$ is context-free.**

2. Let $p$ be the pumping length.

3. **Choose string $S = a^p b^p c^p$.** This string belongs to $L_3$ because $p \leq p \leq p$.

4. Divide $S$ into $uvxyz$ such that $|vy| > 0$ and $|vxy| \leq p$. Similar to $L_2$, $vxy$ can only span at most two blocks of characters ($a$'s, $b$'s, or $c$'s).

- **If $vxy$ is within $a$'s or $b$'s or $c$'s only:** Pumping up or down will change the count of one character type so that the condition $i \leq j \leq k$ may fail. For example, if $v$ and $y$ are only $a$'s, pumping up by suitable choice of $i$ in pumping lemma will violate $i \leq j$. If $v$ and $y$ are only $b$'s or $c$'s, pumping down by choosing $i = 0$ will violate $i \leq j \leq k$ leading to contradiction.
- **If $vxy$ spans $a$'s and $b$'s:** Pumping up increases $a$'s and/or $b$'s. The number of $c$'s remains $p$. Pumping up by suitable choice of $i$ in pumping lemma will violate $j \leq k$.
- **If $vxy$ spans $b$'s and $c$'s:** Pumping *down* ($i = 0$) is effective here. The number of $a$'s remains $p$. Pumping down will violate $i \leq k$.

5. By carefully choosing $i$ (pump up or pump down), a contradiction can be reached. Therefore, $L_3$ is not context-free.

### 3.3 Example: $L_4 = \{ww \mid w \in \{0,1\}^*\}$

**Goal:** Prove $L_4$ is not context-free. This language consists of strings that are a word repeated twice (e.g., 0101, 00110011).

1. **Assume $L_4$ is context-free.**
2. Let $p$ be the pumping length.
3. **Choose string $S = 0^p1^p0^p1^p$.** This string is in $L_4$ (where $w = 0^p1^p$). $|S| = 4p \geq p$.
4. Divide $S$ into $uvxyz$ such that $|vy| > 0$ and $|vxy| \leq p$. Since $|vxy| \leq p$, the segment $vxy$ can only fall into a few regions:
   - **Case 1: $vxy$ is entirely within the first $0^p$ block.** Pumping changes the number of initial 0's, but the structure $0...01^p0^p1^p$ is broken.
   - **Case 2: $vxy$ is entirely within the first $1^p$ block.** Pumping changes the number of first 1's.
   - **Case 3: $vxy$ is entirely within the second $0^p$ block.** Pumping changes the number of second 0's.
   - **Case 4: $vxy$ is entirely within the second $1^p$ block.** Pumping changes the number of second 1's.
   - **Case 5: $vxy$ spans the first $0^p$ and $1^p$ blocks.** (e.g., 0...01...1)
   - **Case 6: $vxy$ spans the first $1^p$ and second $0^p$ blocks.** (e.g., 1...10...0)
   - **Case 7: $vxy$ spans the second $0^p$ and $1^p$ blocks.** (e.g., 0...01...1)

   Consider pumping up ($i = 2$) or down ($i = 0$). If $vxy$ is entirely within one of the four blocks (e.g., $0^p$), pumping will change the length of that block. For instance, if $vxy$ is in the first $0^P$, then $uv^2xy^2z$ will have more 0's at the beginning, breaking the $ww$ pattern. If $vxy$ spans two blocks, for example, the first $1^p$ and second $0^p$ (i.e., $vxy$ is of the form 1...10...0), then pumping up ($i = 2$) will create more 1's and 0's in the middle of the string. The resulting string will no longer be of the form $ww$. For $S = 0^p1^p0^p1^p$, the middle point is between the two $0^p$ blocks. If $vxy$ does not cross this middle point, then pumping will make one half of the

string longer or shorter than the other, or change the character counts in a way that breaks the $ww$ pattern. If $vxy$ *does* cross the middle point, then it must contain parts of both $0^p$ blocks or both $1^p$ blocks, which is impossible given $|vxy| \leq p$.

The key is that $v$ and $y$ must be "close" to each other due to $|vxy| \leq p$. This means they cannot be far apart in the string. In $0^p1^p0^p1^p$, the two $0^p$ blocks are $p$ characters apart, and the two $1^p$ blocks are $p$ characters apart. Since $|vxy| \leq p$, $v$ and $y$ cannot simultaneously affect both occurrences of $0^p$ or both occurrences of $1^p$. Therefore, pumping will always break the equality of the two $w$ parts.

5. This leads to a contradiction. Therefore, $L_4$ is not context-free.

**P.S.:** *The choice of the string $S$ is the most critical part of applying the Pumping Lemma. You need to pick a string that is long enough and structured in such a way that no matter how $vxy$ is chosen (within the lemma's constraints), pumping it will inevitably break the language's defining property.*

### 3.4 Example: $L_5 = \{a^{n!} \mid n \geq 0\}$

This language consists of all strings of length factorial numbers (i.e., $a^{1!}, a^{2!}, a^{3!}, \ldots$).

**Goal:** Prove $L_5$ is not context-free.

**Assume** $L_5$ is context-free.

Let $p$ be the pumping length given by the pumping lemma for context-free languages.

Choose the string $S = a^{p!}$, which is in $L_5$ and has length $p! \geq p$.

By the pumping lemma, $S$ can be written as $uvxyz$ such that $|vy| > 0$ and $|vxy| \leq p$.

Let $d = |v| + |y|$, where $1 \leq d \leq p$.

Consider pumping down with $i = 0$: The new string is

$$uv^0xy^0z = uxz,$$

whose length is

$$|uxz| = |S| - |v| - |y| = p! - d.$$

Now, $p! - d$ will take values in the set $\{p! - 1, p! - 2, \ldots, p! - p\}$, none of which is a factorial for $p \geq 2$. Indeed, the only factorials smaller than $p!$ are $(p-1)!$, $(p-2)!$, and so on, and the gaps between consecutive factorials grow rapidly for large $p$. Thus, $p! - d$ is not equal to $k!$ for any $k \in \mathbb{N}$.

Therefore, $uv^0xy^0z \notin L_5$, which contradicts the pumping lemma.

**So,** $L_5$ is not context-free.

## 4 Next Topic (Turing Machines)

The next topic will introduce **Turing Machines**. Unlike PDAs which have a stack, Turing Machines will have an **infinite tape**, providing even more computational power.

# 5 References

1. P. Linz, An Introduction to Formal Languages and Automata, 6th Edition, Jones & Bartlett Learning, 2016.